Clase 02: Cómputo Monoprocesador

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Martes 10 de Marzo del 2026

Repaso

Cómputo Monoprocesador

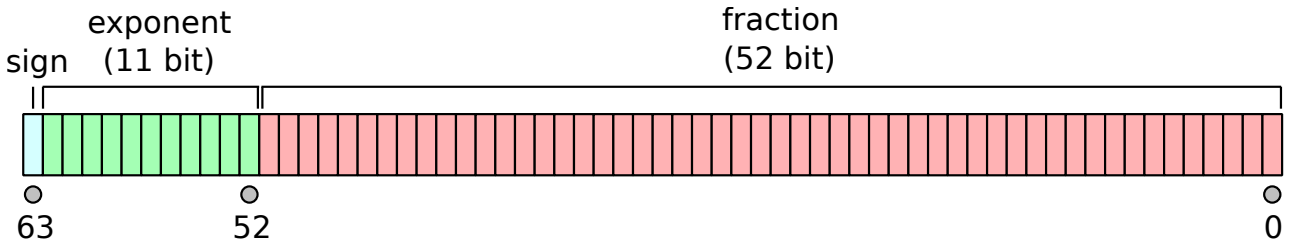
Repaso

Cómputo Monoprocesador

Sistemas de Cómputo Intensivo

Métricas en supercomputadores:

- **FLOP**. Floating Point Operation en precisión doble (8 byte).
- **FLOPS**, o también **FLOP/s**: FLOP por segundo.
- **Byte**. Tamaño de los datos. Precisión doble: 8 byte. Simple: 4 byte.
- **Bit**. Unidad básica. 1 Byte = 8 Bit.



¿Cómo ser más rápido?

Para mayor rapidez, se puede tener

- Más cores en una máquina – *multicore*
  - Dual, Quad, Octa, ...
- Más máquinas con múltiples cores
  - Clusters
  - Memoria compartida o distribuida
- Chips con más procesadores – *manycore* – GPU

¿Paralelismo, distribución, o HPC?

- **Paralelismo**. Cómputo que involucra múltiples tareas ejecutándose simultaneamente como parte de un mismo trabajo.
- **Multiprocesamiento**. Computador ejecutando múltiples trabajos simultáneamente.
- **HPC**. Cómputo de una tarea muy larga, pero que se ejecuta a la mayor velocidad posible.
- **Cómputo distribuido**. Cómputo que involucra múltiples computadores colaborando en una tarea.

¿Por qué es importante?

Ejecutar código eficientemente depende de

- Algoritmo utilizado.
- Aprovechar la arquitectura.

¿Por qué es importante?

Ejecutar código eficientemente depende de

- Algoritmo utilizado.
- Aprovechar la arquitectura.

Tiempo de ejecución puede variar incluso órdenes de magnitud.

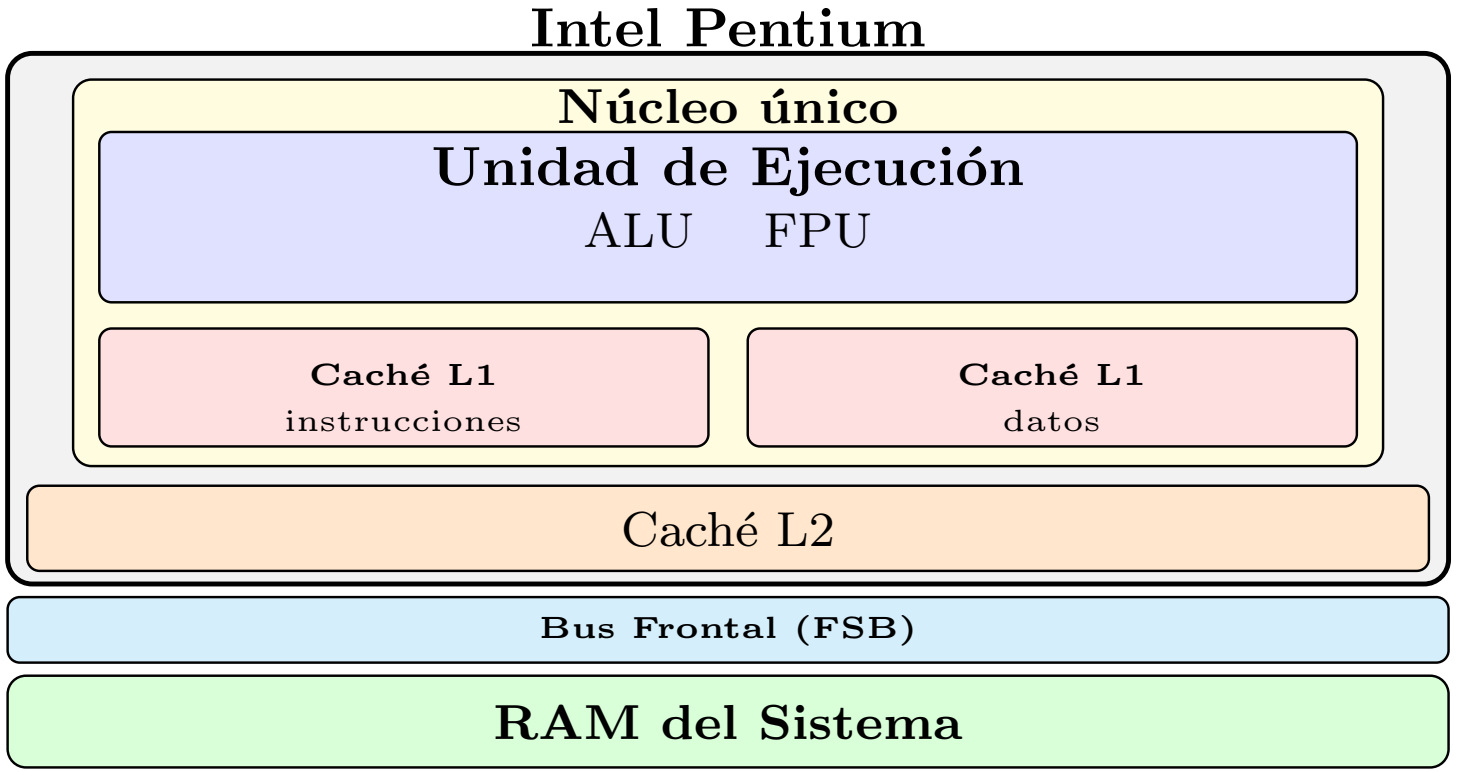
Para algunos problems, **un único procesador** es suficiente.

Repaso

Cómputo Monoprocesador

Un procesador: un solo núcleo

Los primeros procesadores personales tenían un único núcleo. La CPU se comunica con la RAM a través del bus frontal. **Intel Pentium (1993)**:



Antes del silicio: Von Neumann (1945)

El modelo conceptual que rige a Intel Pentium fue definido en **1945** por John von Neumann.

La pregunta que respondió: ¿cómo debe organizarse un computador de propósito general?

Su respuesta sigue siendo la base de **todo procesador moderno**.

Arquitectura Von Neumann

El modelo define dos componentes fundamentales:

- **Memoria**: única y compartida, almacena tanto el programa como los datos.
- **CPU**: ejecuta instrucciones en secuencia siguiendo el *Program Counter*.
  - Ciclo: *Fetch* $\rightarrow$ *Decode* $\rightarrow$ *Execute* $\rightarrow$ *Write-back*.
  - Modelo *Control flow*: el flujo lo dictan las instrucciones.

Consecuencia clave: los programas son datos. Pueden modificarse, pasarse como argumentos, generarse en tiempo de ejecución.

Instrucciones de bajo nivel

La CPU ejecuta **instrucciones de bajo nivel**: operaciones elementales sobre registros y memoria.

Operación	Dirección Operandos	Dirección Resultado

- Los operandos y resultados se referencian por **dirección de memoria**.
- En la práctica, las direcciones son registros: pequeñas celdas dentro del procesador, de acceso muy rápido.
- La RAM es órdenes de magnitud más lenta que los registros; minimizar accesos a memoria es clave para el rendimiento.

Ejemplo: C → Assembler

¿Cómo se ve esto en la práctica? Un compilador traduce código C a instrucciones de bajo nivel:

```c
void store(double *a, double *b, double *c){
   *c = *a + *b;
}
```

Compilar: `gcc -O2 -S -o - ejemplo.c`

```
        .file "ejemplo.c"
        .text
        .p2align 4
        .globl  store
        .type store, @function
store:
.LFB0:
.cfi_startproc
movsd (%rdi), %xmm0     # Load *a to %xmm0
addsd (%rsi), %xmm0     # Load *b and add to %xmm0
movsd %xmm0, (%rdx)     # Store result to *c
ret
.cfi_endproc
```

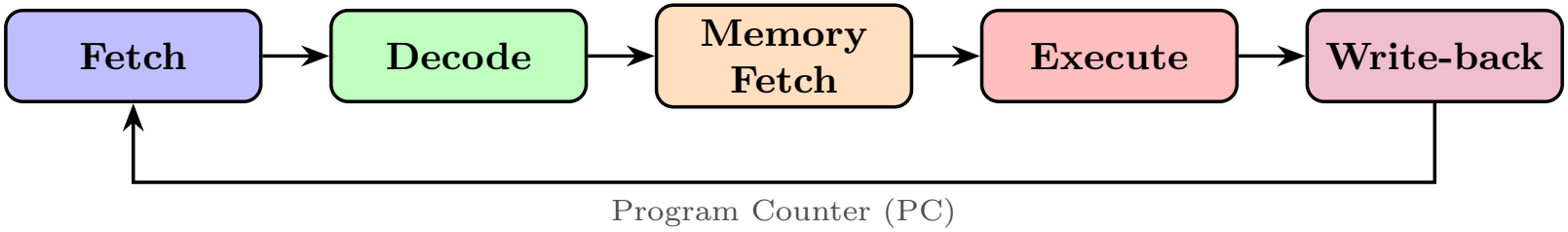
El ciclo de instrucción

Ciclo:

- ***Fetch***.
  Cargar instrucción en el procesador de acuerdo a *Program Counter*.
- **Decode**.
  Determinar qué operación es y qué operandos necesita.
- **Memory fetch**.
  Traer operandos desde memoria a registros (si es necesario).
- **Execute**.
  Ejecuta la operación, leyendo y escribiendo registros.
- **Write-back**.
  Guarda el resultado en memoria (si es necesario).

Más allá del ciclo básico

El funcionamiento básico es



pero los computadores modernos introducen mecanismos para hacer más trabajo por ciclo:

ILP: Instruction Level Parallelism

- **Múltiples unidades de ejecución**. Operaciones independientes ocurren en paralelo.
- **Pipelining**. Varias instrucciones se procesan al mismo tiempo en distintas etapas.

Unidades de ejecución

El modelo Von Neumann supone una sola unidad de ejecución. Los procesadores modernos tienen varias, todas dentro de un mismo core:

- **ALU** (*Arithmetic Logic Unit*): aritmética entera y operaciones lógicas (and, or, shift, . . .).
- **FPU** (*Floating Point Unit*): operaciones de punto flotante (suma, multiplicación, división).
- **AGU** (*Address Generation Unit*): calcula direcciones de memoria para loads y stores.
- **Load/Store Unit**: ejecuta los accesos a memoria.
- **Branch Unit**: evalúa saltos condicionales y trabaja junto al predictor de saltos.

Unidades de punto flotante (FPU)

Nos enfocaremos en las **FPU**: el cuello de botella aritmético en cómputo científico.

- Los procesadores modernos tienen **múltiples FPUs** por core.
- Pueden ser unidades **add**, **mul**, o **fma** (cada una con distinto costo y capacidad).
- Cada FPU puede operar sobre **múltiples datos a la vez** (ancho SIMD).
- El rendimiento máximo del core depende directamente de cuántas FPUs tiene y qué tan anchas son.

DAXPY

DAXPY (*Double-precision A·X Plus Y*): operación de referencia en álgebra lineal numérica.

$$y_i \leftarrow a \cdot x_i + y_i, \quad i = 1, \ldots, n$$

- a: escalar. x, y: vectores de doubles.
- Una multiplicación + una suma por elemento.

Operaciones afines

La misma operación $a \times b + c$ es el núcleo aritmético de operaciones más complejas:

- **DAXPY** (vector): $y \leftarrow ax + y$
- **GEMV** (matriz $\times$ vector): $y \leftarrow Ax + y$
- **GEMM** (matriz $\times$ matriz): $C \leftarrow AB + C$
- **Convoluciones y tensores**: operaciones centrales en *deep learning*.

Optimizar multiply-add impacta toda la cadena.

Tipos de FPU y ancho SIMD

Los procesadores tienen distintos tipos de FPUs y distintas cantidades de ellas:

- **add** : solo suma.
- **mul** : solo multiplica.
- **fma** : ejecuta $a \times b + c$ en una sola instrucción.

Cada unidad tiene un **ancho** (*width*): cuántos `double` procesa a la vez.

- Ancho 1: 1 resultado `double` por ciclo.
- Ancho 4: 4 resultados `double` por ciclo (SIMD[1] de 256 bits).

[1] Single Instruction Multiple Data

Fused Multiply-Add (FMA)

Una FMA es una FPU que ejecuta $a \times b + c$ en una sola instrucción, en vez de requerir una unidad *add* y una *mul* separadas.

```
// Sin FMA: dos instrucciones, dos redondeos
double t = a * b;      // redondeo 1
double r = t + c;      // redondeo 2

// Con FMA: una instruccion, un redondeo
double r = fma(a, b, c); // a*b + c exacto, luego redondea
```

- Más rápido : una instrucción en vez de dos.
- Más preciso : $a \times b$ se mantiene exacto internamente antes de sumar c.

Ejemplo: MIPS R10000

MIPS R10000 (1996): `add+mul:` 1×1. DAXPY para vectores de 8 elementos:

$$\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_7 \end{bmatrix} \leftarrow a \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_7 \end{bmatrix} + \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_7 \end{bmatrix}$$

- Aritmética: 8 mul + 8 add. Con pipelining, la mul de $i+1$ solapa con la add de $i \Rightarrow$ **8 ciclos**.
- Memoria:
  `load x[i]` + `load y[i]` + `store y[i]` = 3 operaciones $\times$ 8 = **24 ciclos**.

El cuello de botella es la memoria, no la aritmética.

Evolución de procesadores

Ciclos para DAXPY en distintos procesadores. Formato: `add` + `mul` + `fma`, cada una `cantidad` $\times$ `ancho`:

Processor	year	add/mult/fma units (count $\times$ width)	daxpy cycles (arith / load-store)
MIPS R10000	1996	$1 \times 1 + 1 \times 1 + 0$	8/24
Alpha EV5	1996	$1 \times 1 + 1 \times 1 + 0$	8/12
IBM Power5	2004	$0 + 0 + 2 \times 1$	4/12
AMD Bulldozer	2011	$2 \times 2 + 2 \times 2 + 0$	2/4
Intel Sandy Bridge	2012	$1 \times 4 + 1 \times 4 + 0$	2/4
Intel Haswell	2014	$0 + 0 + 2 \times 4$	1/2

Fuente: Hager & Wellein (2010). La tendencia continúa en procesadores posteriores.

Velocidad de Reloj

El procesador opera sincronizado por un **reloj** (*clock*):

- **Frecuencia** f: número de ciclos por segundo. Se mide en GHz.
- **Período de ciclo** $\tau = 1/f$: duración de un ciclo. Se mide en ns.
- Ejemplo: un procesador a $3{,}5\ \mathrm{GHz}$ tiene $\tau \approx 0{,}286\ \mathrm{ns}$ por ciclo.

Cada operación tarda un número fijo de ciclos. La velocidad real depende de:

- La frecuencia del reloj.
- El número de instrucciones completadas por ciclo (**IPC**).
- $\mathrm{FLOPS} = f \times \mathrm{IPC} \times \#\mathrm{FPUs}$

Peak Performance

Con *pipelining*, cada FPU produce un resultado por ciclo (asintóticamente). El rendimiento máximo teórico de un core es:

$$\text{Peak Performance} = \text{clockspeed} \times \#\text{FPUs} \times \text{ancho}$$

Intel Haswell (3,5 GHz, 2 FMA de ancho 4):

$$3{,}5 \times 10^9 \times 2 \times 4 \times 2 = 56 \text{ GFLOP/s por core}$$

(el factor $\times 2$ porque cada FMA cuenta como 2 FLOP: una mul y una add.)

- Es un límite teórico: en la práctica nunca se alcanza.
- **Eficiencia**: $\eta = \frac{\text{FLOP/s medidos}}{\text{Peak Performance}}$.

Instruction Level Parallelism (ILP)

Los procesadores modernos **no ejecutan una instrucción a la vez**.

Explotan el paralelismo a nivel de instrucción para hacer más trabajo por ciclo:

- **Pipelining**.
  Distintas instrucciones avanzan simultáneamente por etapas del ciclo.
- ***Branch prediction* y *speculative execution***.
  El procesador predice saltos y ejecuta instrucciones anticipadamente.
- ***Out-of-order execution***.
  Instrucciones independientes se reordenan para evitar esperas.
- ***Prefetching***.
  Datos se cargan en caché antes de ser solicitados.

Instruction Level Parallelism (ILP)

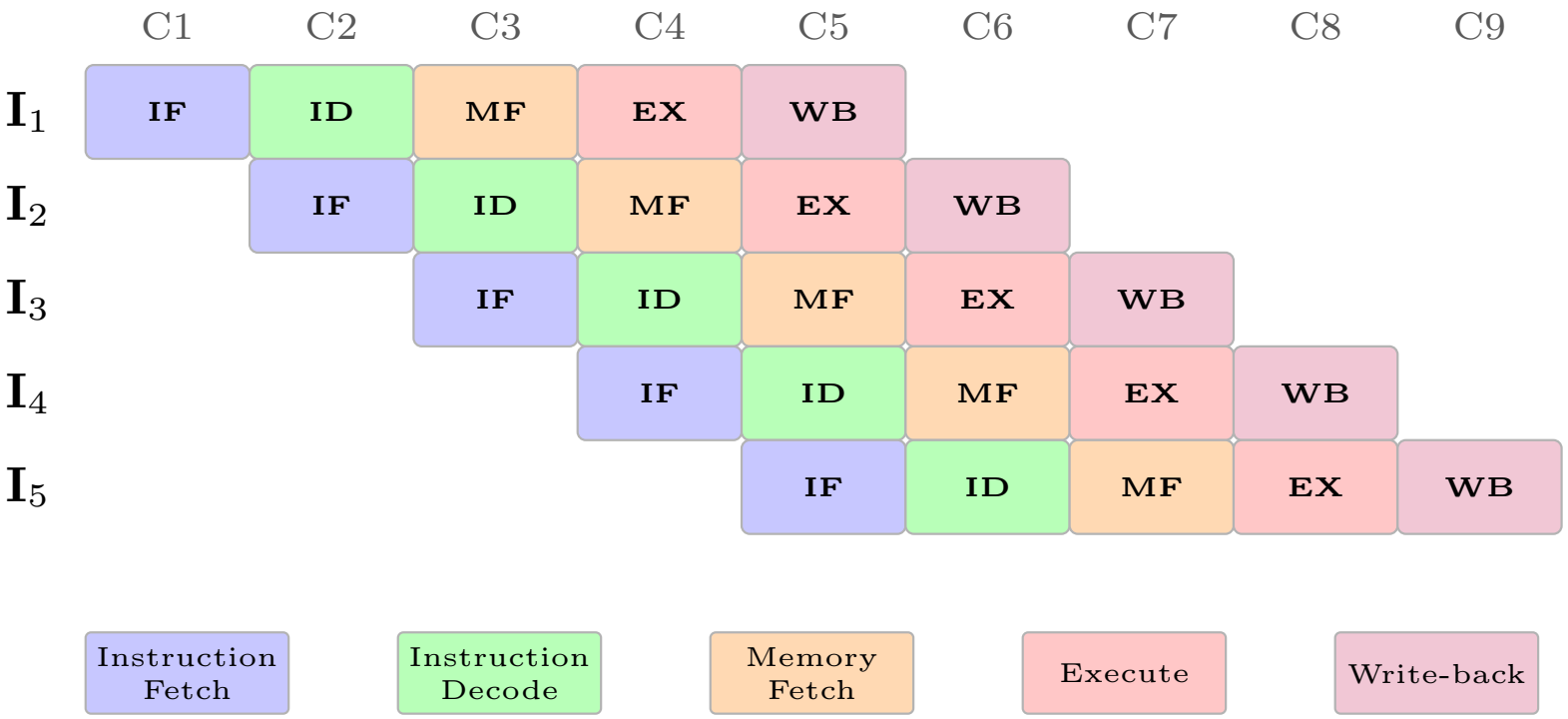
Los procesadores modernos **no ejecutan una instrucción a la vez**.

Explotan el paralelismo a nivel de instrucción para hacer más trabajo por ciclo:

- **Pipelining**.
  Distintas instrucciones avanzan simultáneamente por etapas del ciclo.
- ***Branch prediction* y *speculative execution***.
  El procesador predice saltos y ejecuta instrucciones anticipadamente.
- ***Out-of-order execution***.
  Instrucciones independientes se reordenan para evitar esperas.
- ***Prefetching***.
  Datos se cargan en caché antes de ser solicitados.

Pipelining: ejecución solapada

Varias instrucciones se procesan **simultáneamente** en distintas etapas.



- Cada instrucción tarda ℓ ciclos (5 etapas).
- Con *pipelining*: a partir del ciclo ℓ, sale **un resultado por ciclo**.
- Requiere que las instrucciones sean independientes entre sí.

¿Cuánto gano con un *pipelining*?

FPU Tradicional: Obtener n resultados toma $T(n) = n \times \ell \times \tau$

- ℓ : número de etapas
- τ : período de ciclo.
- Tasa de resultados es $n/T(n)$, $r_{\text{serial}} = \frac{n}{T(n)} = \frac{1}{\ell \times \tau}$.

Pipelined FPU: $T(n) = (s + \ell + n - 1) \times \tau$.

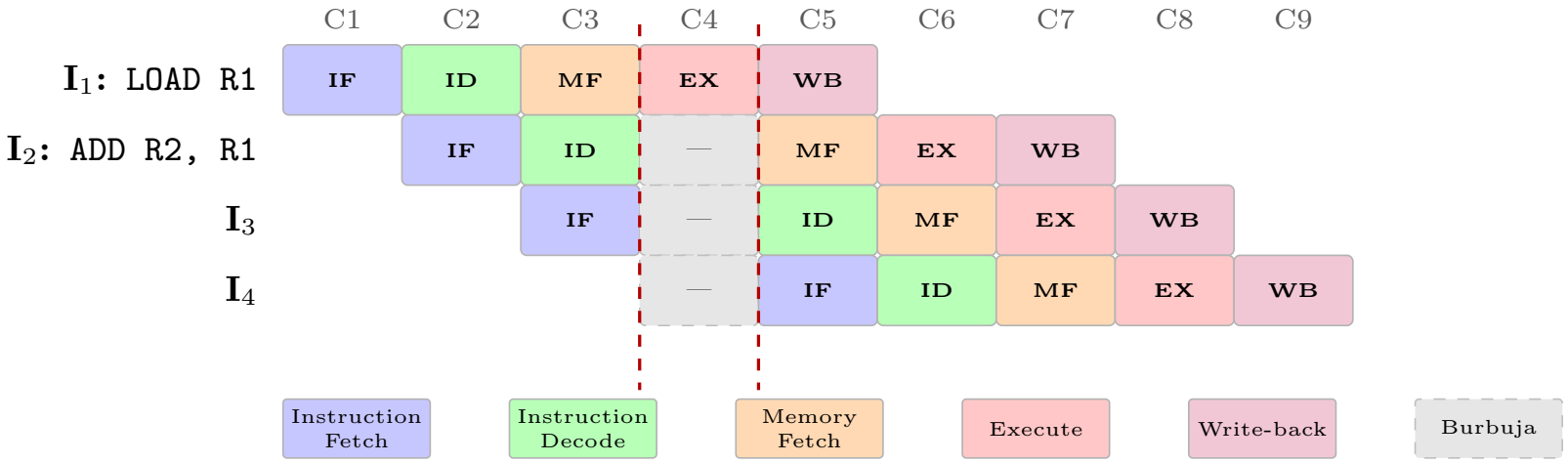
- s : ciclos de preparación antes del primer resultado (*setup*).
- La primera operación recorre todo el *pipeline*.
- Después, las siguientes terminan una a la vez.

Ejemplo: $\ell = 5$, $\tau = 1$, $s = 0$, $n = 100$ operaciones.

- Serial: $T = 100 \times 5 = 500$ ciclos. Tasa: $\frac{1}{5}$ resultado/ciclo.
- Pipelined: $T = 5 + 100 - 1 = 104$ ciclos $\approx 5\times$ más rápido.
- Tasa asintótica ($n \to \infty$): 1 resultado/ciclo.

Pipelining: burbujas por dependencia de datos

Si I_2 necesita el resultado de I_1, el pipeline se **detiene**:



- Una burbuja (*stall*) se propaga a todas las instrucciones siguientes.
- El compilador puede reordenar instrucciones para evitar dependencias.

Recurrencias en el pipeline

Las instrucciones en el *pipeline* deberían ser independientes

- $a_i \leftarrow b_i + c_i$

Recurrencias en el pipeline

Las instrucciones en el *pipeline* deberían ser independientes

- $a_i \leftarrow b_i + c_i$
- $a_{i+1} \leftarrow a_i b_i + c_i$

¿Cómo ejecutar esto en un pipeline?

```
1      for(i = 0; i < 1000; i++){
2         a[i+1] = a[i] * b[i] + c[i];
3      }
```

Recurrence unrolling

La solución es desarrollar algebraicamente la recurrencia para saltar un paso:

$$a[i+2] = a[i] \cdot \underbrace{b[i]\, b[i+1]}_{B_i} + \underbrace{c[i]\, b[i+1] + c[i+1]}_{C_i}$$

Recurrence unrolling

La solución es desarrollar algebraicamente la recurrencia para saltar un paso:

$$a[i+2] = a[i] \cdot \underbrace{b[i]\, b[i+1]}_{B_i} + \underbrace{c[i]\, b[i+1] + c[i+1]}_{C_i}$$

Esto genera dos cadenas independientes que el pipeline puede solapar:

Cadena par: $a[0] \to a[2] \to a[4] \to \cdots$

Cadena impar: $a[1] \to a[3] \to a[5] \to \cdots$

- Cada cadena tiene la misma latencia que antes, pero avanzan **en paralelo**.
- Con k cadenas se necesita precomputar B_i y C_i.
- Técnica general: *recurrence unrolling*.

Punto flotante y asociatividad

- El procesador no transforma álgebra.
- El compilador puede hacerlo con `-ffast-math`.
- Las operaciones de punto flotante no son asociativas.

Ejemplo: sea `a = 1e20`, `b = -1e20`, `c = 1.0`

```
(a + b) + c =  0.0   + 1.0   =  1.0

a + (b + c) =  1e20  + (-1e20 + 1.0)
            =  1e20  + (-1e20)
            =  0.0
```

`1.0` se pierde por redondeo al sumarse a `-1e20`.

Instruction Level Parallelism (ILP)

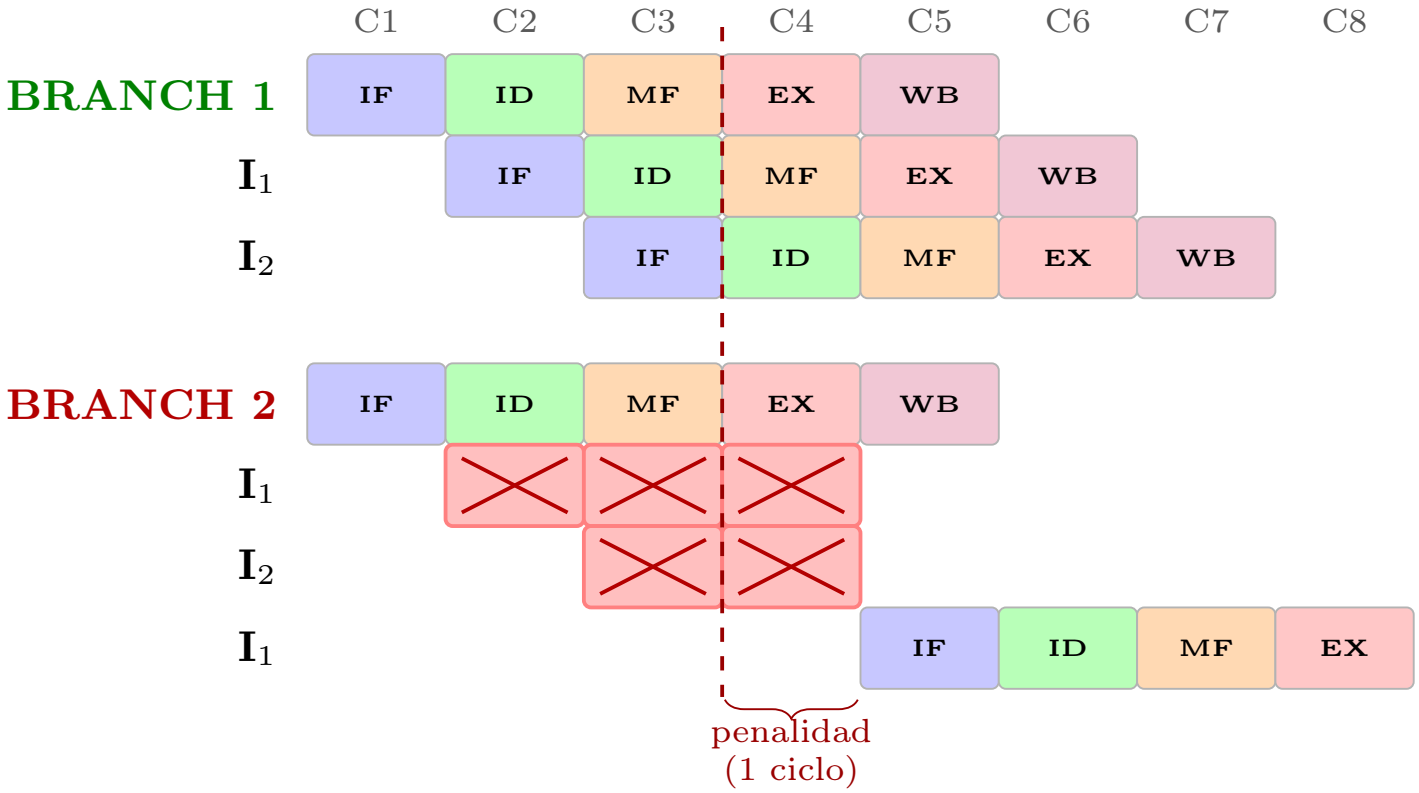
Los procesadores modernos **no ejecutan una instrucción a la vez**.

Explotan el paralelismo a nivel de instrucción para hacer más trabajo por ciclo:

- **Pipelining**.
  Distintas instrucciones avanzan simultáneamente por etapas del ciclo.
- ***Branch prediction* y *speculative execution***.
  El procesador predice saltos y ejecuta instrucciones anticipadamente.
- ***Out-of-order execution***.
  Instrucciones independientes se reordenan para evitar esperas.
- ***Prefetching***.
  Datos se cargan en caché antes de ser solicitados.

Branch Prediction y Speculative Execution

El procesador **predice** el resultado de un salto condicional y ejecuta instrucciones anticipadamente:



- Predicción correcta : el pipeline no se interrumpe.
- Predicción incorrecta : las instrucciones erróneas se cancelan (*flush*), penalidad de varios ciclos.

Branch Prediction: estrategias

Al encontrar un `if`, el procesador debe predecir qué rama ejecutará **antes** de evaluarla. Distintas estrategias:

- **Estática**. Siempre predice que se entra al `if`, o que no se entra.
- **1-bit**. Recuerda el último resultado y predice lo mismo.
- **2-bit saturating counter**. Requiere dos fallas consecutivas para cambiar la predicción. Robusto para loops.
- **Correlating / 2-level**. Usa historial de los últimos N saltos para detectar patrones.
- **Tournament / TAGE** (procesadores modernos). Combina múltiples predictores con distintos largos de historial.

Instruction Level Parallelism (ILP)

Los procesadores modernos **no ejecutan una instrucción a la vez**.

Explotan el paralelismo a nivel de instrucción para hacer más trabajo por ciclo:

- **Pipelining**.
  Distintas instrucciones avanzan simultáneamente por etapas del ciclo.
- ***Branch prediction* y *speculative execution***.
  El procesador predice saltos y ejecuta instrucciones anticipadamente.
- ***Out-of-order execution***.
  Instrucciones independientes se reordenan para evitar esperas.
- ***Prefetching***.
  Datos se cargan en caché antes de ser solicitados.

Out-of-Order Execution: motivación

```
1  for (int i = 0; i < N; i++) {
2      sum  += a[i];
3      prod *= b[i];
4      div  = c[i] / d[i];
5  }
```

- La división toma ~20 ciclos; suma y multiplicación toman ~1–3.
- Las tres operaciones son independientes entre sí.
- Mientras la división de la iteración i está en curso, el procesador adelanta la suma y multiplicación de la iteración $i + 1$.
- ***Out-of-order execution***: ejecutar instrucciones fuera del orden del programa, respetando las dependencias.

Out-of-Order Execution: mecanismo interno

- ***Reorder Buffer* (ROB)**: guarda instrucciones en orden de programa. El commit ocurre en orden aunque la ejecución no.
- ***Reservation stations***: cada instrucción espera hasta que sus operandos estén listos y luego se despacha.
- **Renombramiento de registros**: elimina dependencias falsas (WAR, WAW) usando un banco de registros físicos más grande.
- El tamaño del ROB define la **ventana de ejecución** (típicamente 200–300 instrucciones).

Out-of-Order Execution: tipos de dependencias

Hay tres tipos de dependencias entre instrucciones:

- **RAW** (*Read After Write*) : dependencia **verdadera**.
  I_1 produce un valor que I_2 necesita leer. **No se puede eliminar**.
- **WAR** (*Write After Read*) : *anti-dependencia* (falsa).
  I_2 escribe un registro que I_1 todavía debe leer. Se elimina con **renombramiento**.
- **WAW** (*Write After Write*) : *dependencia de salida* (falsa).
  I_1 e I_2 escriben el mismo registro. Se elimina con **renombramiento**.

El renombramiento convierte WAR y WAW en operaciones sobre registros distintos, dejando solo las dependencias RAW que limitan el paralelismo real.

Instruction Level Parallelism (ILP)

Los procesadores modernos **no ejecutan una instrucción a la vez**.

Explotan el paralelismo a nivel de instrucción para hacer más trabajo por ciclo:

- **Pipelining**.
  Distintas instrucciones avanzan simultáneamente por etapas del ciclo.
- ***Branch prediction* y *speculative execution***.
  El procesador predice saltos y ejecuta instrucciones anticipadamente.
- ***Out-of-order execution***.
  Instrucciones independientes se reordenan para evitar esperas.
- ***Prefetching***.
  Datos se cargan en caché antes de ser solicitados.

Prefetching

La ejecución fuera de orden no oculta la latencia de memoria (~200 ciclos). La solución: traer los datos antes de que se necesiten.

- **Hardware**: detecta strides constantes y precarga automáticamente. Falla con acceso irregular.
- **Software**: el programador emite *prefetch* explícitos. Es útil cuando el hardware no detecta el patrón.
- El *prefetch* excesivo contamina la memoria temporal.

Límites del ILP

Las técnicas de ILP presentan problemas:

- **Dependencias RAW**: no se pueden eliminar. Cadenas de dependencias largas dejan las unidades de ejecución ociosas.
- **Ventana de ejecución finita**: el ROB tiene ~200–300 entradas. Dependencias fuera de la ventana no se explotan.
- **Mispredictions**: un salto mal predicho vacía el pipeline y desperdicia decenas de ciclos.
- ***Memory wall***: si el dato no está en caché, el procesador espera ~200 ciclos sin avanzar, sin importar cuánto ILP haya.

El cuello de botella es la memoria. Para seguir escalando se necesita paralelismo explícito: múltiples hilos, múltiples cores.

¿Qué pasa en Python?

Todo lo visto ocurre a nivel del **hardware**. El intérprete de Python introduce overhead que rompe la regularidad que el hardware necesita.

¿Cómo escribir código Python que sí aproveche el hardware?

Python: loop vs NumPy

```python
import numpy as np
N = 1000
a = 2.0
x = np.random.rand(N)
y = np.random.rand(N)
```

Loop Python: cada iteración pasa por el intérprete.

```python
for i in range(N):
    y[i] = a * x[i] + y[i]
```

NumPy: una sola llamada delega a código C compilado.

```python
y = a * x + y
```

- El compilador de C genera instrucciones SIMD y permite *pipelining*.
- NumPy aprovecha la regularidad del hardware: acceso secuencial, operaciones independientes.